Final Year Project · 2025 / 26

Cycle-level Profiling of the GPU Cache Hierarchy

Sub-instruction profiling of PTX cache modifiers on Ampere & Turing

Pierre Khoury · BSc CS with AI · University of Sussex

Supervisor: Martin Berger

What we'll cover

16 sections · ≈15 min talk + 5 min Q&A

01 Why AI Kernels Are Slow

- AI kernels spend most cycles moving bytes, not FLOPs.
- 9 of 12 DL kernels memory-bound on RTX 3060 Ti.
- Only big GEMMs cross to the compute side.

Takeaway

Attack per-load cost, not per-FLOP.

Kernel	SM %	DRAM %
Add (Addition · a+b)	4	**95**
ReLU (Rectified Linear Unit)	7	**93**
LayerNorm (Layer Normalization)	17	**93**
Reduction Σ (Sum reduction)	8	**83**
Softmax (Soft Maximum)	37	**73**
Conv 28^2 1→32 (Convolution)	28	**70**
Conv 56^2 64→128 (Convolution)	54	54
GEMM 256^3 (General Matrix Multiply)	37	37
Attn (Scaled Dot-Product Attention · SDPA)	51	44
FlashAttn (Flash Attention, Triton impl.)	21	20
GEMM 1024^3 (General Matrix Multiply)	**72**	60
GEMM 4096^3 (General Matrix Multiply)	**75**	58

memory-bound (top) · mixed (middle) · compute-bound (bottom)

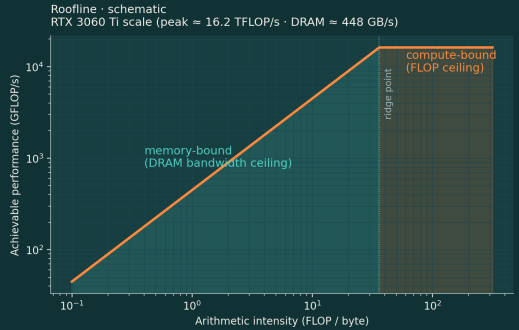
SM % = % of peak *compute* · DRAM % = % of peak *bandwidth* (independent peaks)

02 Memory- vs Compute-Bound · Why NCU Matters

- Left of ridge = memory-bound; right = compute-bound.
- Source can't tell you. WS, cache state, modifier all matter.
- NCU is ground truth: SM-pipe %, DRAM-pipe %, sectors.
- No counter = optimise blind.

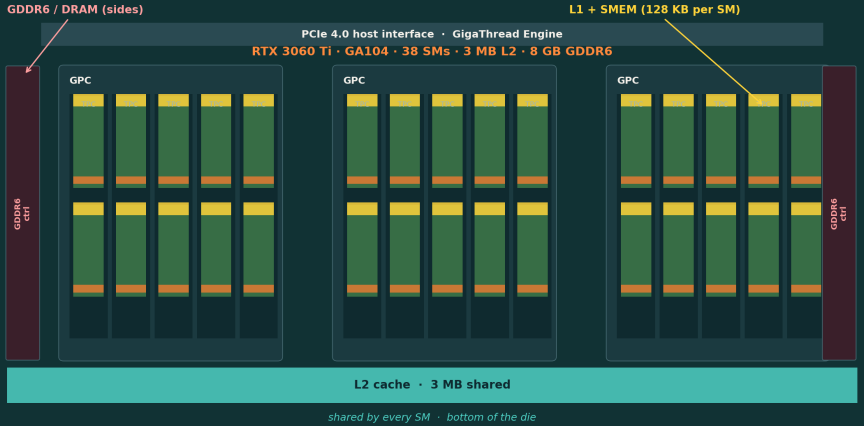
Why this slide exists

Every later slide assumes we can tell memory-bound from compute-bound. NCU counters are the only ground truth · clock64 alone can't.



Roofline figure: schematic to RTX 3060 Ti scale (peak ≈ 16.2 TFLOP/s, DRAM ≈ 448 GB/s) ref:

03 GPU Die Layout · Where Each Tier Lives



GA104 die · GPC = Graphics Processing Cluster · SM = Streaming Multiprocessor · SMEM = Shared Memory · GDDR6 = Graphics DDR6

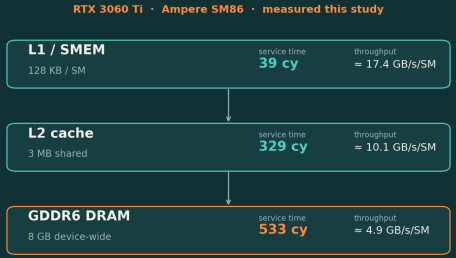
03 GPU Architecture

- Ampere SM86 (Streaming Multiprocessor arch. 8.6) · 38 SMs (Streaming Multiprocessors) @ 1.68 GHz.
- L1/SMEM (Level 1 cache / Shared Memory): 128 KB per SM.
- L2 (Level 2 cache): 3 MB shared.
- DRAM (Dynamic RAM): 8 GB GDDR6 (Graphics DDR6).
- Modifier picks which tier serves a load.

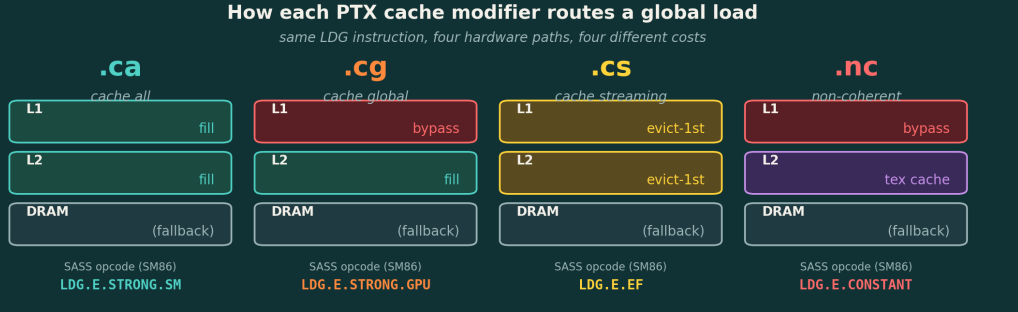
What the latencies cost

L1: 39 cy (cycles) · L2: 329 cy · DRAM: 533 cy

8.4× ratio L1→L2 · biggest lever in the hierarchy.



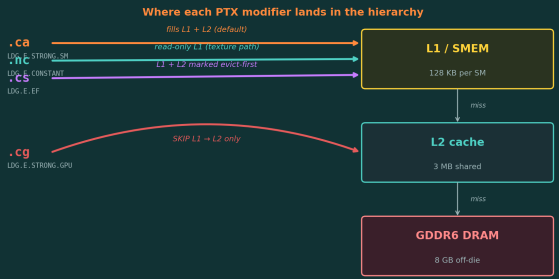
04 What Each Cache Modifier Actually Does



Verified on RTX 3060 Ti · every modifier disassembles to a distinct SASS opcode via cuobjdump.

Working-set governs which tier serves the load. Modifier choice governs how that tier handles the line.

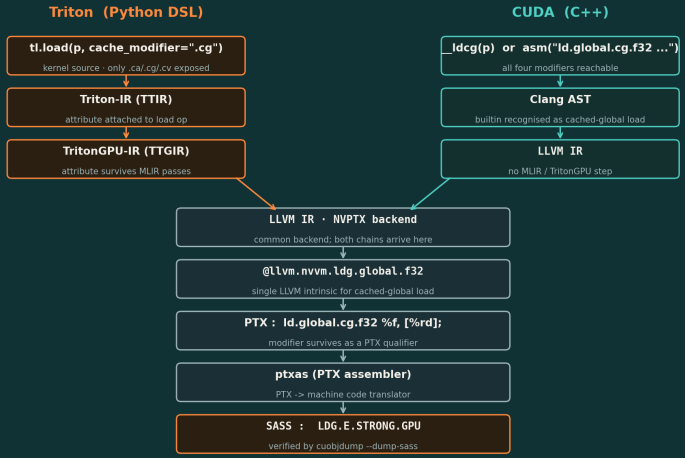
04 The 4 PTX Cache Modifiers



One PTX line, four routes. Same instruction, just swap the suffix.

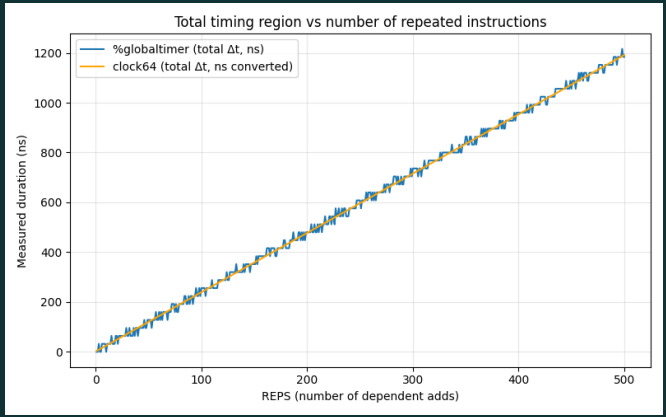
- `.ca` is the default — straight into L1, fills L1 and L2.
- `.nc` also lands in L1 but through the read-only texture path — the compiler only emits this when it can prove the data isn't being written.
- `.cs` fills both tiers, but every line gets stamped evict-first so it gets pushed out fast — for streaming data with no reuse.
- `.cg` is the one that bypasses L1 entirely. The arrow goes around the L1 box and lands in L2.

04 From Source to SASS · How the Modifier Lowers



Both languages converge at the NVPTX intrinsic; the modifier survives as a PTX qualifier

05 How It Started · Timing Dependent FADDs



- X-axis: chain length, 0 to 500 dependent `FADD`s.
- Y-axis: total elapsed time, nanoseconds.
- Two timers overlaid: `%globaltimer` (blue) and `clock64` (orange, cycles → ns).
- Both linear, slope ≈ 2.4 ns/add.
- Timers agree across all chain lengths.

06 Methodology · Three Instruments Side by Side

- `clock64` times one LDG (in-kernel, sub-instruction).
- NCU PMU reports tag hits, sector counts and bytes after the run.
- CUDA events time the whole kernel host-side (effective GB/s).
- Different physical signals · agreement licenses, disagreement informs.

Why all three

`clock64` sees timing, not tier. NCU sees tier, not timing. CUDA events scale to kernel-wide sweeps (§10 decision map).

axis	clock64 *in-kernel cycle counter*	NCU PMU *post-run HW counters*	CUDA events *host-side wall-clock*
granularity	one LDG at a time	kernel total or warp average	whole kernel launch
what it sees	cycles between two clock reads	tag hits, sector counts, bytes	ms per launch · effective GB/s
invasiveness	two extra cycles per bracket	zero, replays the kernel	zero, host-side timing
blind spot	tier identity	bank stalls	per-instruction cost; tier identity
when it wins	modifier ranking, bank stalls	tier confirmation, sector traffic	WS×warps sweeps, effective bandwidth
how we use it	primary timing for §8 results	validates clock64 tier by tier	§10 decision-map heatmap

Three instruments, three angles. clock64 + NCU triangulate per-LDG cost; CUDA events scale to whole-kernel sweeps.

Disagreement is the signal: the only w = 6 split was the L1 bank stall (NCU 98.46% hits unchanged; clock64 +34% cycles).

07 Nano-Benchmarking

- Two `CS2R` reads bracket one `LDG`.
- `R0:W5` = scoreboard barrier.
- `FFMA` = first consumer; pins the load before `t1`.
- Δ/N = per-LDG cycles.
- 1 in flight $\rightarrow$ latency; many $\rightarrow$ throughput.

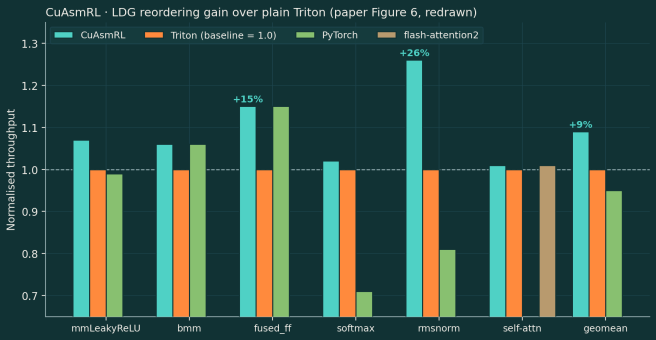
Why kernel-level isn't enough

NCU averages thousands of loads into one number. A 290-cycle L2 fill bleeding into the next bracket can reverse a modifier ranking · invisible at kernel level.

08 Similar Work · Why Per-Instruction Cost Matters

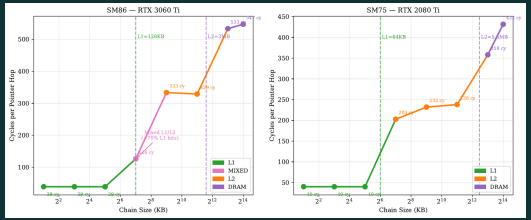
- **CuAsmRL**: RL agent reorders SASS `LDG`s (and tunes control codes); reward = CUDA-event kernel time → up to **+26%** on memory-bound kernels.
- **KPerfIR**: compiler IR that inserts per-instruction profiling primitives at instruction level · kernel-granularity profilers are too coarse for automated optimisation.
- Both *consume* per-instr cost. Neither *measures* it.
- We're the missing instrument.

The leverage
If reordering LDGs gives +26%, the cost of any LDG is worth knowing first.



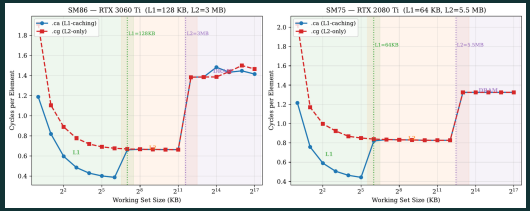
09 Results · Hierarchy Resolved Two Ways

1. Pointer-chase latency · chain size sweep



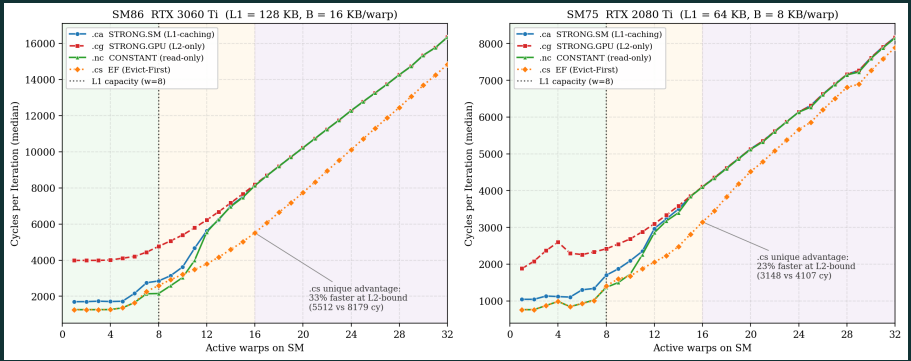
- One `LDG` per pointer hop; chain size = working set per thread.
- Three flat plateaus at ∼**40 cy** (L1), ∼**220 cy** (L2), ∼**535 cy** (DRAM).
- Knees align with L1 (128 KB) and L2 (3 MB) on SM86; same shape on SM75.

2. Throughput · working-set size sweep



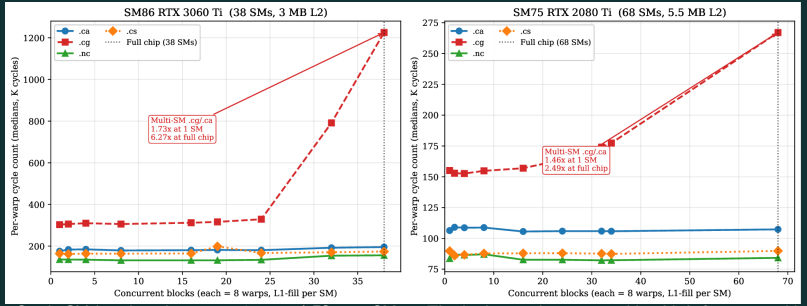
- `.ca` (L1-caching) vs `.cg` (L2-only) swept; same three regions appear.
- Latency falls, then jumps, then falls, then jumps · once per tier transition.
- Two unrelated experiments converge on the *same* L1/L2/DRAM structure.

09 Results · All Four Modifiers, Side by Side



- Sub-L1 ($w < 8$): `.ca`/`.nc`/`.cs` hit L1 (∼1.7 K cy); `.cg` is pinned to L2 (∼4 K cy).
- At **w=8** (128 KB WS), L1 spills and the modifier ranking collapses.
- $w \geq 16$: all four converge · DRAM dominates, modifier choice is moot.
- Same shape on SM86 (L1=128 KB) and SM75 (L1=64 KB); knee tracks L1 capacity.

09 Results · Multi-SM Scaling · .cg Penalty at Full Chip



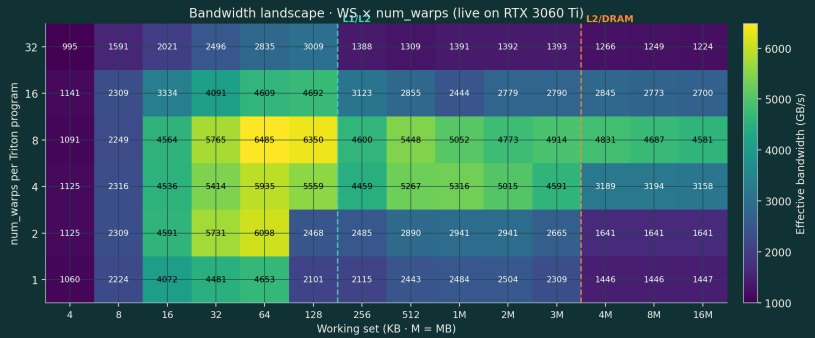
- Single-SM benchmarks see `.cg/.ca` = **1.73**× on SM86. That's the number you would publish from a one-block experiment.
- Light up all 38 SMs and the ratio grows to **6.27**× · every SM bypasses L1 and slams shared L2; contention scales nonlinearly.
- `.ca`, `.cs`, `.nc` stay flat · L1 absorbs each SM's working set locally, no inter-SM contention.
- SM75 (2080 Ti) shows the same direction: 1.46× at one SM, 2.49× at full chip · less extreme because L2 is larger (5.5 MB vs 3 MB) and there are more SMs to amortise.
- **Implication:** `.cg` is not just "slower than `.ca`" · it's a *scaling liability*. The penalty you measure on one SM understates the cost in production.

10 NCU Bank-Stall · When the Two Instruments Disagree

- NCU and `clock64` agree at $w{=}1, 2, 4$ and $w{\geq}8$.
- At $w{=}6$ L1 hit unchanged at 98.43 %, `clock64` **+34 %**.
- Same tier, different cost · L1 bank conflict.

warps	WS (KB)	L1 hit %	L2 hit %	DRAM sectors	clock64 cycles
1	16	98.43	68.4	1 000	1 701
2	32	98.43	95.7 *(noise)*	1 436	1 705
4	64	98.43	53.9	2 204	1 711
6	**96**	**98.43**	**60.4**	**3 240**	**2 290 (+34 %)**
8	128	78.95	90.5	4 256	2 388
16	256	0.00	98.1	9 292	8 158
32	512	0.00	98.1	17 364	16 270
32	4 096	0.00	0.14	**8 382 328**	–

11 The Decision Map · WS × Num_warps → Bandwidth



- Peak **6485 GB/s** at nw = 8, WS = 64 KB.
- **6.4**× spread · wrong config beats wrong modifier.
- Tier lines at 128 KB and 3 MB.
- nw = 32 collapses · occupancy / bank pressure.

12 Live Demo

1. **Hierarchy sweep** · 3 plateaus visible.
2. **Warp sweep** · BW drops 45% nw=4→32.
3. **NCU contradiction** · hit rate flat, cost varies.
4. **Modifier sweep** · `.cg` 4–6× slower than `.ca`.
5. **Decision landscape** · live heatmap.

What's running

Triton + a single CUDA bracket cell. 34 cells. ∼4 min on the SSH'd 3060 Ti.

Transparent

`ncu -csv` subprocess parses live · metric IDs visible.

If it fails

Cached outputs as fallback.

13 Relevance · the Per-Instruction Wave

- Vendor + research converging on the same granularity.
- Cluster at **per-instruction** since 2020.
- Methodology transfers across ISAs: same bracket pattern.

The principle

Bracket-one-instruction works on any ISA with a free-running cycle counter: RDTSC, clock64, PMCCNTR, rdcycle.

- **Intel** `TPAUSE` (2020) · cycle-precise micro-pause baked into x86; the vendor put a per-instruction timing primitive in the ISA itself.
- **Tasnadi** (CUDA, 2024) · brackets individual CUDA instructions with `clock64`; kernel-level metrics hide where the cycles actually go.
- **KPerfIR** (2025) · compiler IR with first-class per-instruction profiling primitives; makes bracketing systematic, not ad-hoc inline asm.
- **CuAsmRL** (CGO '25) · RL reorders SASS `LDG`s, **+26 %** on memory-bound kernels; per-instruction scheduling beats block-level tuning.
- **This project** (2026) · brackets each PTX cache modifier across L1/L2/DRAM; the missing modifier × working-set cost map.

2020 onwards: vendor + research both converge on per-instruction.

14 Supervision · From CPU Concepts to GPU Profiling

Recurring meeting themes

- **Prefetching** · how a CPU hides memory latency by issuing the load before the consumer needs it.
- **Pointer-chasing** · latency-bound workloads where each load gates the next; throughput tricks don't help.
- **The "mega-instruction"** · take a whole pointer chain and treat it as *one* timed primitive for memory loading.

How it shifted to GPU

- CPU's `RDTSC` timing → GPU's `clock64` + `CS2R` bracket.
- The pointer-chase latency benchmark on slide 12 *is* the mega-instruction idea applied to one `LDG` at a time.
- Sub-instruction bracketing — the methodology of this whole deck — started as a CPU framing in those meetings.

What I took away

Bracket one instruction, time it precisely, sweep the structure around it. CPU framing; GPU implementation.

15 Limitations and Scope

- **Granularity** · warp-level. Not per-load.
- **Synthetic** · single-LDG bench. Real workloads need pattern match.
- **Triton 3.2** · `.cs` dead zone is version-specific.
- **Not factorial** · block size / occupancy controlled, not swept.
- **Single-GPU** · no NVLink, no multi-die.
- **`.nc` = CUDA only** · live demo shows 3 of 4 modifiers.

Still solid

SASS mapping (`cuobjdump`-verified). Three-tier structure (cross-instrument).

16 Future Work · H100 and Beyond

Datacenter GPUs · where this matters most

- Hopper: longer latencies → modifier matters *more*.
- L1 **256 KB / SM** → wider actionable window.
- L2 **50 MB** → L2-bound covers most LLM WS.
- HBM3 + TMA → do cp.async semantics still hold? Open.
- Methodology transfers · `clock64`, NCU, cuobjdump all exist.

Compiler · upstream the modifier choice

- Patch Triton's `tl.load` to expose all four modifiers. Today only `.ca`/`.cg`/`.cv` reach PTX; `.cs` and `.nc` are absent (issue `triton-lang/triton#5946`).
- Auto-pick the modifier from working-set × warp shape using this project's cost map.
- Validate on real Triton kernels: GEMM-K-streamed, attention K/V loads.
- Submit upstream + reproducible benchmark.

Q&A

Thank you.

Pierre Khoury
University of Sussex · 2025 / 26

demo: `demo/demo.ipynb`